PROJECT PLAN

Supply Chain Late Delivery Predictor

End-to-End Machine Learning & Generative AI Project

Author	Fernando González
Program	Data Science & Machine Learning Bootcamp
Dataset	DataCo SMART SUPPLY CHAIN (Kaggle)
Date	April 2026

Project Overview

This project delivers an end-to-end machine learning solution for predicting late delivery risk in a global supply chain context. Using the DataCo SMART SUPPLY CHAIN dataset, the project covers the full data science lifecycle: data acquisition and cleaning, exploratory analysis, feature engineering, model training and comparison, hyperparameter optimisation with Optuna, and deployment as an interactive Streamlit application augmented with Generative AI recovery actions via the OpenAI API. The final model — a tuned XGBoost classifier — achieved 91% accuracy on training data and 68% on holdout data, identifying over 90% of high-risk orders before they leave the warehouse.

✓ Completed	⚙ In Progress	○ Planned

PHASE 01 — Setup & Data Acquisition

✓ COMPLETED

Established the project foundation including version control, environment configuration, and initial data exploration to understand the raw dataset structure.

- ✓ Create GitHub repository with folder structure (/data, /notebooks, /src, /app, /docs)
- ✓ Set up virtual environment and requirements.txt
- ✓ Download DataCo SMART SUPPLY CHAIN dataset from Kaggle
- ✓ Initial data exploration — shape, dtypes, nulls, basic statistics

PHASE 02 — Data Preparation

✓ COMPLETED

Defined a clean, leak-free feature set by eliminating post-shipment columns, handling missing values, encoding categorical variables, and validating the target variable.

- ✓ Define the pre-shipment feature set (eliminate data leakage columns)
- ✓ Confirm and finalize target variable (Late_delivery_risk)
- ✓ Handle missing values and outliers
- ✓ Type casting and feature encoding (categorical → numerical)
- ✓ Feature selection and correlation analysis

PHASE 03

Exploratory Data Analysis (EDA)

✓ COMPLETED

Conducted in-depth analysis of delivery performance patterns across regions, shipping modes, and customer segments. Exported clean dataset for Tableau visualisation.

- ✓ Univariate and bivariate analysis of key features
- ✓ Delivery performance analysis by region, shipping mode, and customer segment
- ✓ Statistical analysis — distributions, correlations, variance
- ✓ Export clean dataset for Tableau

PHASE 04

Tableau Dashboard

✓ COMPLETED

Built an operational Tableau dashboard to visualise late delivery patterns by region and shipping mode, providing business stakeholders with an accessible analytical view.

- ✓ Connect cleaned dataset to Tableau
- ✓ Build operational visuals (late delivery rate by region, shipping mode, etc.)
- ✓ Publish dashboard

PHASE 05

Machine Learning

✓ COMPLETED

Trained and compared four model architectures — Logistic Regression, XGBoost, LightGBM, and a Neural Network. Applied Optuna hyperparameter tuning to the best-performing model, reducing missed high-risk orders from 7,908 to 1,743.

- ✓ Temporal train/holdout split (last 10% of orders as holdout)
- ✓ Baseline model — Logistic Regression with StandardScaler
- ✓ Advanced models — XGBoost (200 trees, depth 6) and LightGBM
- ✓ Deep Learning comparison — feedforward neural network (128→64→32, early stopping)
- ✓ Hyperparameter tuning with Optuna (50 trials, maximise ROC-AUC)
- ✓ Final model: XGBoost — 458 trees, depth 10, learning rate 0.239

- ✓ Model evaluation and selection (F1, Precision, Recall, ROC-AUC)
- ✓ Feature importance analysis — Shipping Mode and Scheduled Days identified as top drivers

PHASE 06

Generative AI Component

✓ COMPLETED

Integrated OpenAI GPT-4o-mini to generate contextual, order-specific recovery plans for high-risk deliveries. Prompt engineering ensured outputs were concise and operationally relevant.

- ✓ Design prompt template using model prediction context (risk score + key features)
- ✓ Build script that takes a high-risk order and generates recovery actions via LLM
- ✓ Test and refine outputs for business relevance

PHASE 07

Streamlit Application

✓ COMPLETED

Deployed an interactive web application combining real-time model prediction with AI-generated recovery actions. Users can input order details and receive an immediate risk assessment and recommended response plan.

- ✓ Build Streamlit UI with order input form (shipping mode, region, product, etc.)
- ✓ Integrate tuned XGBoost model for real-time risk scoring
- ✓ Connect OpenAI API for automated recovery action generation on high-risk orders
- ✓ Deploy application to Streamlit Cloud

PHASE 08

Wrap-Up & Documentation

✓ COMPLETED

Final documentation, code cleanup, and presentation preparation to produce a portfolio-ready deliverable showcasing the full project lifecycle.

- ✓ Write README.md (goals, methodology, results, how to run)
- ✓ Clean and document all code and notebooks
- ✓ Build final presentation slides (20 slides, business and technical audience)
- ✓ Final commit, push, and repository polish

Key Results

91%	68%	7,908→1,743	< 1 sec
Training Accuracy	Holdout Accuracy	Missed Risk Orders Fixed	AI Recovery Response

Technology Stack

Data & Analysis	Python, Pandas, NumPy, Matplotlib, Seaborn, Tableau
Machine Learning	Scikit-learn, XGBoost, LightGBM, TensorFlow/Keras, Optuna
Generative AI	OpenAI API (GPT-4o-mini), Prompt Engineering
Application	Streamlit, Python-dotenv, Joblib
Dev & Deployment	GitHub, Virtual Environment (venv), Streamlit Cloud